

Project Check-in 2

Econ 5293

Nishant Yamujala

Relationship between a small or medium sized firm's decision to hire H1B STEM workers and its subsequent profitability

1. Data Cleaning

For this project, I relied on two primary data sources, and additional helper data sources to create the panel needed. First, I relied on the “H1-B Employer Data hub” which contains firm-by-firm data by year on H1B initial approvals and denials, and continuing approvals and denials. An immigrant worker first applies for a H1B visa through their employer(initial), and if the application is approved by USCIS, the worker can extend their visa for an additional three-year period. After the maximum six-year term on H1B, the worker's employer will have to file for a green card if they want to retain the worker. The “continuing” approval and denial columns track this extension process. Since there are country caps for green cards, some workers from countries like India and China can remain on the H1B visa for years, and it's unclear if the continuing approval column tracks that (More careful analysis will have to be done to cross-check this with data from the BLS PERM applications or I-140 approvals). Each individual Excel file had to be downloaded and had the filename format: “h1b_datahubexport-XXXX.csv” where XXXX stands for the year. The data are available on the website from 2009 – 2023, and after downloading the Excel files, they were all merged into a single Excel file called: “h1b_all_years.csv”. Additionally, another combined file was created with tech companies filtered (Two-digit NAICS codes 31-33, 51, and 54, since the H1B dataset only has two-digit NAICS codes without additional granularity). This final H1B filtered dataset was used for the analysis(“h1b_tech_filtered.csv”)

Obtaining the firm level financial data was tricky and Compustat from Wharton Data research services was used. The Compustat interface allows one to filter firms by number of employees(emp), or several other query variables. My first instinct was to use the Russell 2000 index, but my subscription didn't include licensed Russell data, so the choice was made to keep firms ranked 1001–3000 by market cap each year. I kept firms for which historical NAICS (naicsh) starts with 334 (computer/electronic manufacturing), 511 (software publishers), 518 (data processing/hosting), or 5415 (computer systems design services). That left 626 unique firms, for the period from 2010-2023.

The next obstacle was matching the firms in the H1B combined dataset with the compustat dataset. Merging the two datasets was a little challenging as the company names weren't identical between the H1B and Compustat datasets. To aid with matching, the RapidFuzz library was used (<https://github.com/rapidfuzz/RapidFuzz>) with some available code templates. Since the final objective of the project is to use Difference in Differences techniques to look at firm level performance subject to a shock like the 2017 buy American hire American executive order (The executive branch enjoys great control over immigration enforcement), additional indicator variables were created. The final dataset has 626 total unique firms for the period from 2010-2023.

2. Summary Statistics

The final dataset has 45 columns with variables identifying the firm, firm size statistics, profitability statistics, investment metrics, H1B data information including approval, denial data (initial and continuing), and other calculated metrics including the ones needed for DiD analysis.

```
=====
DATASET OVERVIEW
=====
```

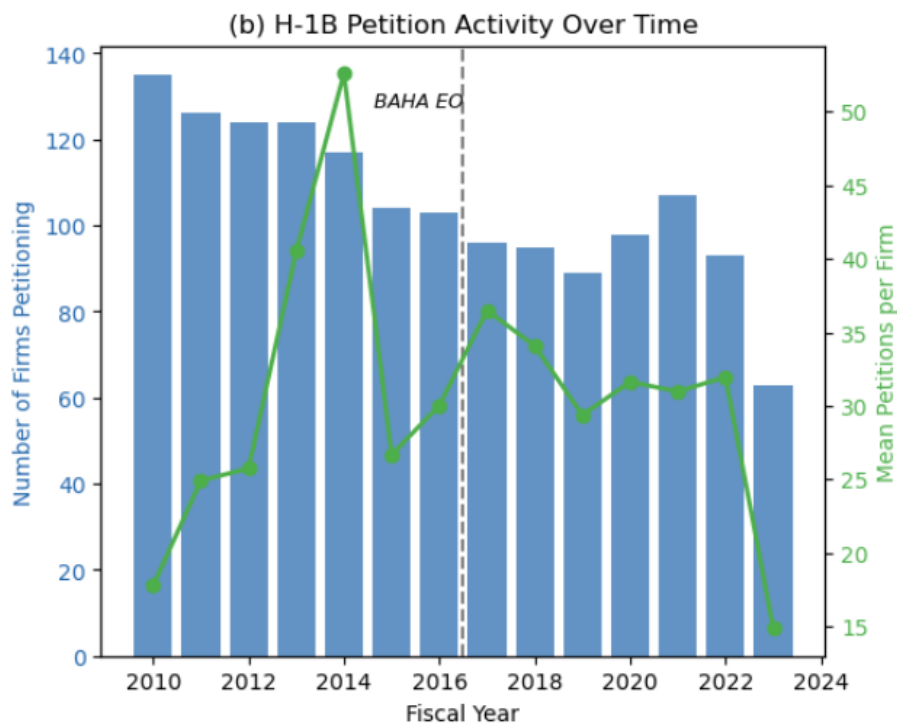
```
Observations (firm-years): 2928
Unique firms:              626
Year range:                2010 - 2023
```

The summary statistics of the data is as follows:

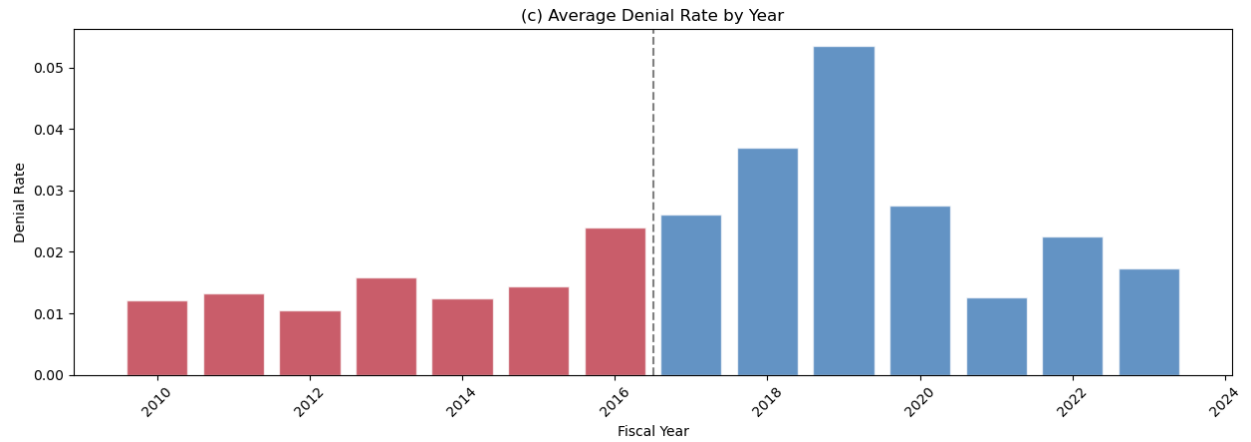
Variable	N	Mean	SD	P25	Median	P75
Employees (000s)	2886	8.996	22.288	1.394	3.042	7.223
Revenue (\$M)	2927	1728.669	3007.109	425.546	854.637	1820.440
Total Assets (\$M)	2928	2392.275	3258.869	670.817	1387.646	2754.438
Market Cap (\$M)	2928	3008.433	2066.113	1439.734	2350.587	4023.602
Net Income (\$M)	2927	31.016	336.248	-20.283	44.400	119.631
EBITDA (\$M)	2927	222.919	352.827	46.406	135.300	301.096
Return on Assets	2927	0.016	0.148	-0.017	0.035	0.076
Operating Margin	2922	-0.441	18.958	0.073	0.150	0.216
R&D Expense (\$M)	2502	139.147	182.230	45.172	90.501	168.138
R&D / Revenue	2499	0.231	2.115	0.066	0.135	0.205
Capital Expenditure (\$M)	2925	89.024	280.698	11.311	27.215	63.181
Cash Holdings (\$M)	2928	462.428	555.572	147.474	294.850	548.099
H-1B Petitions	2928	12.081	65.216	0.000	0.000	6.000
H-1B Approvals	2928	11.676	62.400	0.000	0.000	6.000
H-1B Denials	2928	0.406	3.366	0.000	0.000	0.000
H-1B Approval Rate	1130	0.979	0.056	0.985	1.000	1.000
H-1B per Employee	2928	0.003	0.009	0.000	0.000	0.002

3. Preliminary Modeling

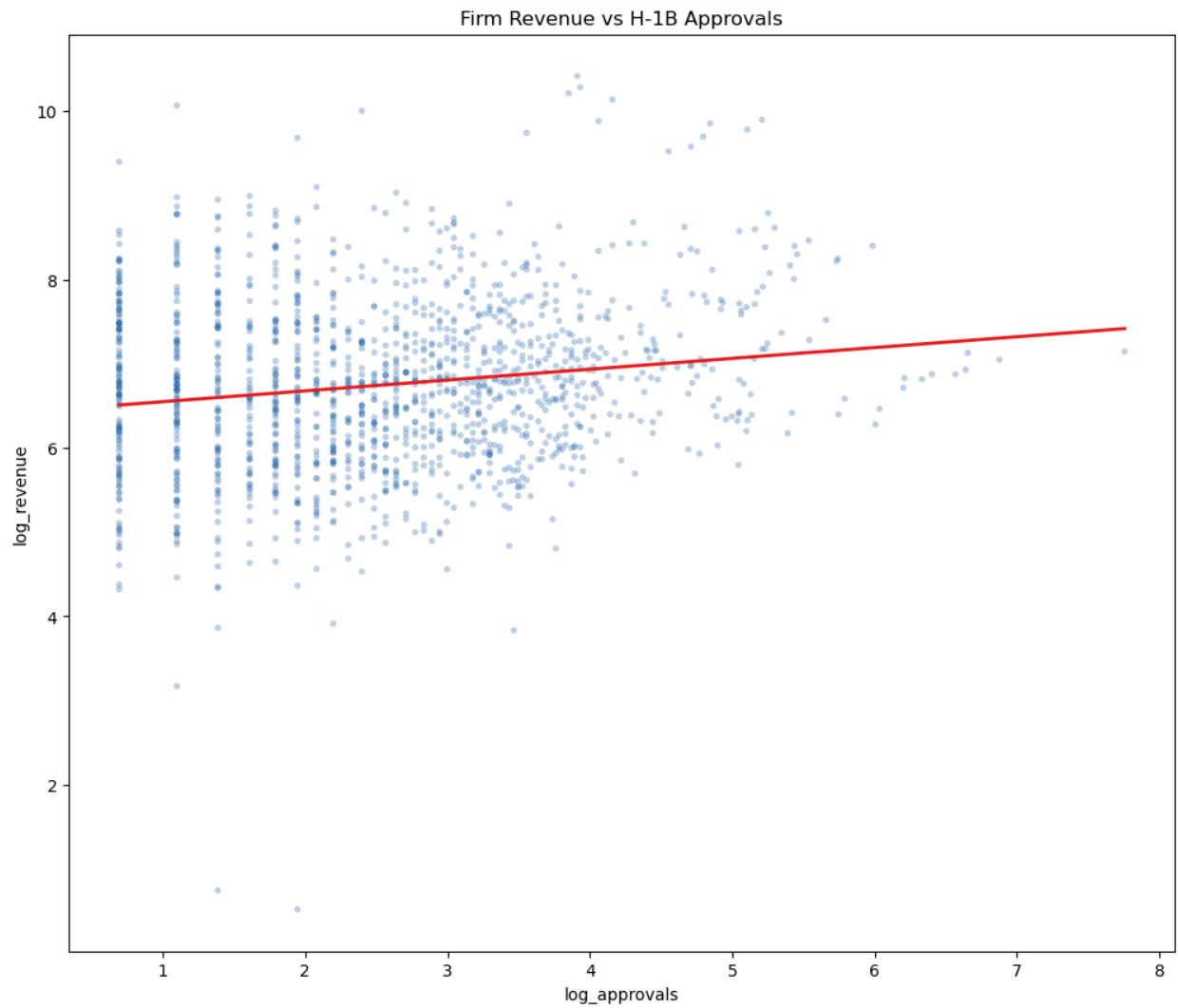
It's clear that there was a drop in the number of H1B petitions from 2016-2020 due to the build in America and buy American executive order



There were also a larger number of denials after 2016,



An initial simple regression was performed with firm revenue vs. number of H1B approvals



OLS Regression Results						
=====						
Dep. Variable:	log_rev	R-squared:	0.024			
Model:	OLS	Adj. R-squared:	0.023			
Method:	Least Squares	F-statistic:	36.29			
Date:	Fri, 20 Mar 2026	Prob (F-statistic):	2.14e-09			
Time:	21:33:45	Log-Likelihood:	-2080.1			
No. Observations:	1474	AIC:	4164.			
Df Residuals:	1472	BIC:	4175.			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	6.4194	0.060	106.957	0.000	6.302	6.537
log_approvals	0.1286	0.021	6.024	0.000	0.087	0.170
=====						
Omnibus:	65.732	Durbin-Watson:	1.773			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	209.489			
Skew:	0.037	Prob(JB):	3.24e-46			
Kurtosis:	4.845	Cond. No.	7.21			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

A 10% increase in H1B approvals is associated with a roughly 1.3% increase in revenue and this is a statistically significant result. Since the R^2 is 0.024, the H1B approvals explain only 2.4% of the variation in revenue. This makes sense since companies are complicated entities with profitability being affected by several factors.

4. Preliminary Conclusions

So far, companies with increased H1B approvals are indeed more profitable, but clearly this analysis isn't complete. This could simply be an indication that larger firms or firms in certain sectors are more profitable and hence hire more employees, including H1B workers. Further analysis is needed with controls.

Furthermore, it makes sense to include a treatment and control group of companies in the same industry with similar structure and profits. These companies can be separated by firms that hire more H1B workers, and firms that don't. Only then can a more causal relationship be gleaned from the data. This further analysis and possible inclusion of additional data will be part of the future check-ins and the final project. My objective is to perform a DiD analysis which also tracks the impact of the immigration restrictions that followed after 2016.

ProjectCheckIn_2_NY

March 20, 2026

```
[20]: import pandas as pd
import glob

files = glob.glob('h1b_datahubexport-*.csv')
h1b = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)
h1b.to_csv('h1b_all_years.csv', index=False)

frames = []

for f in files:
    temp = pd.read_csv(f, dtype=str)
    temp.columns = temp.columns.str.strip()
    rename_map = {
        'Initial Approval': 'Initial Approvals',
        'Initial Denial': 'Initial Denials',
        'Continuing Approval': 'Continuing Approvals',
        'Continuing Denial': 'Continuing Denials',
    }
    temp = temp.rename(columns=rename_map)
    frames.append(temp)

h1b = pd.concat(frames, ignore_index=True)

# Verify all years present
print(h1b['Fiscal Year'].value_counts().sort_index())

# Convert numeric columns
for col in ['Initial Approvals', 'Initial Denials', 'Continuing Approvals',
            'Continuing Denials']:
    h1b[col] = pd.to_numeric(h1b[col], errors='coerce').fillna(0).astype(int)

# Filter to tech NAICS
h1b['NAICS'] = h1b['NAICS'].astype(str).str.strip()
h1b_tech = h1b[h1b['NAICS'].isin(['31', '32', '33', '51', '54'])].copy()

h1b_tech['total_approvals'] = h1b_tech['Initial Approvals'] +
    h1b_tech['Continuing Approvals']
```

```

h1b_tech = h1b_tech[h1b_tech['total_approvals'] > 0]

print(f"\nFull H1B rows: {len(h1b)}")
print(f"\nTech-filtered rows: {len(h1b_tech)}")
print(f"\nYears in filtered data:")
print(h1b_tech['Fiscal Year'].value_counts().sort_index())

h1b_tech.to_csv('h1b_tech_filtered.csv', index=False)

```

C:\Users\nisha\AppData\Local\Temp\ipykernel_18968\703875043.py:5: DtypeWarning: Columns (3) have mixed types. Specify dtype option on import or set low_memory=False.

```
h1b = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)
```

Fiscal Year

2009	68919
2010	55429
2011	62874
2012	56222
2013	56079
2014	56595
2015	48544
2016	53129
2017	49786
2018	55666
2019	59441
2020	55239
2021	60806
2022	59983
2023	33332

Name: count, dtype: int64

Full H1B rows: 832044

Tech-filtered rows: 457149

Years in filtered data:

Fiscal Year

2009	35063
2010	27640
2011	32421
2012	29285
2013	29188
2014	31073
2015	27746
2016	30233
2017	28534
2018	30949
2019	32119


```

2020    31472
2021    35749
2022    35099
2023    20578
Name: count, dtype: int64

```

```
[3]: !pip install rapidfuzz
```

```
Collecting rapidfuzz
```

```

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```

```

Downloading rapidfuzz-3.14.3-cp310-cp310-win_amd64.whl.metadata (12 kB)
Downloading rapidfuzz-3.14.3-cp310-cp310-win_amd64.whl (1.5 MB)
----- 0.0/1.5 MB ? eta -:-:--
----- 0.0/1.5 MB ? eta -:-:--
----- 0.3/1.5 MB ? eta -:-:--
----- 0.5/1.5 MB 1.2 MB/s eta 0:00:01
----- 0.8/1.5 MB 1.2 MB/s eta 0:00:01
----- 1.0/1.5 MB 1.2 MB/s eta 0:00:01
----- 1.3/1.5 MB 1.2 MB/s eta 0:00:01
----- 1.5/1.5 MB 1.1 MB/s eta 0:00:00

```

```

Installing collected packages: rapidfuzz
Successfully installed rapidfuzz-3.14.3

```

```

[21]: # installed rapidfuzz
import numpy as np
import re
from rapidfuzz import fuzz, process

panel = pd.read_csv('compustat_r2000_tech_panel.csv')
h1b = pd.read_csv('h1b_tech_filtered.csv')

print(f"Compustat panel: {len(panel)} firm-years, {panel['gvkey'].nunique()} firms")
print(f"H-1B data: {len(h1b)} rows, {h1b['Employer'].nunique()} unique employers")
print(f"Compustat years: {panel['fyear'].min()}--{panel['fyear'].max()}")
print(f"H-1B fiscal years: {h1b['Fiscal Year'].min()}--{h1b['Fiscal Year'].max()}")

def clean_company_name(name):
    """
    Standardize company names for fuzzy matching.
    Strips legal suffixes, abbreviations, punctuation, and extra whitespace.
    """

```

```

if pd.isna(name):
    return ''
name = str(name).upper().strip()

# Remove common legal/corporate suffixes and abbreviations
removals = [
    r'\bINC\b\.\?', r'\bCORP\b\.\?', r'\bCORPORATION\b', r'\bINCORPORATED\b',
    r'\bLLC\b', r'\bLTD\b\.\?', r'\bLIMITED\b',
    r'\bCO\b\.\?', r'\bCOMPANY\b',
    r'\bLP\b', r'\bLLP\b', r'\bPLC\b',
    r'\bGROUP\b', r'\bHOLDINGS\b', r'\bHOLDING\b',
    r'\bINTERNATIONAL\b', r'\bINTL\b',
    r'\bTECHNOLOGIES\b', r'\bTECHNOLOGY\b', r'\bTECH\b',
    r'\bSOLUTIONS\b', r'\bSOLNS\b',
    r'\bSYSTEMS\b', r'\bSYS\b',
    r'\bSERVICES\b', r'\bSVCS\b', r'\bSVC\b',
    r'\bAMERICA\b', r'\bAMERICAS\b',
    r'\bUSA\b', r'\bUS\b',
    r'\bNA\b',
    r'\s*-\s*CL\s*[A-Z]\b', # share class designations (-CL A, -CL B)
    r'\bTHE\b',
    r'\bENTERPRISES\b', r'\bENTERPRISE\b',
    r'\bLABORATORIES\b', r'\bLABS\b', r'\bLAB\b',
    r'\bINDUSTRIES\b',
]

for pattern in removals:
    name = re.sub(pattern, '', name)

# Remove punctuation and collapse whitespace
name = re.sub(r'[\w\s]', '', name)
name = re.sub(r'\s+', ' ', name).strip()

return name

# Get unique firms from each dataset
firms = panel[['gvkey', 'conm']].drop_duplicates('gvkey').copy()
firms['conm_clean'] = firms['conm'].apply(clean_company_name)
print(f"Compustat firms to match: {len(firms)}")

h1b_employers = h1b[['Employer']].drop_duplicates().copy()
h1b_employers['employer_clean'] = h1b_employers['Employer'].
    ↪ apply(clean_company_name)
h1b_clean_list = h1b_employers['employer_clean'].unique().tolist()
print(f"H-1B unique employers: {len(h1b_clean_list)}")

# Run fuzzy matching (score_cutoff=70 to capture candidates for manual review)
print("Running fuzzy matching (this may take a few minutes)...")

```

```

results = []
for i, row in firms.iterrows():
    match = process.extractOne(
        row['conm_clean'],
        h1b_clean_list,
        scorer=fuzz.token_sort_ratio,
        score_cutoff=70
    )
    if match:
        matched_clean, score, idx = match
        original_h1b = h1b_employers[
            h1b_employers['employer_clean'] == matched_clean
        ]['Employer'].iloc[0]
        results.append({
            'gvkey': row['gvkey'],
            'conm': row['conm'],
            'conm_clean': row['conm_clean'],
            'h1b_employer': original_h1b,
            'h1b_employer_clean': matched_clean,
            'match_score': score,
        })
    else:
        results.append({
            'gvkey': row['gvkey'],
            'conm': row['conm'],
            'conm_clean': row['conm_clean'],
            'h1b_employer': None,
            'h1b_employer_clean': None,
            'match_score': 0,
        })

crosswalk = pd.DataFrame(results)
crosswalk.to_csv('name_crosswalk.csv', index=False)

print(f"\nMatching summary:")
print(f"  Score >= 90 (high confidence):  ")
    ↳ {len(crosswalk[crosswalk['match_score'] >= 90])}
print(f"  Score 80-90 (needs review):    ")
    ↳ {len(crosswalk[(crosswalk['match_score'] >= 80) & (crosswalk['match_score']
    ↳ < 90)])}
print(f"  Score 70-80 (likely wrong):    ")
    ↳ {len(crosswalk[(crosswalk['match_score'] >= 70) & (crosswalk['match_score']
    ↳ < 80)])}
print(f"  No match:                      ")
    ↳ {len(crosswalk[crosswalk['match_score'] == 0])}

# Auto-accept score >= 90

```

```

# Manually rescue legitimate matches in the 80-90 range
# (These were identified by inspecting the borderline output)
manual_good_conm = [
    'EXLSERVICE HOLDINGS INC',
    'BOOZ ALLEN HAMILTON HLDG CP',
    'BROADRIDGE FINANCIAL SOLUTNS',
    'DOUBLEVERIFY HOLD INC',
    'ADVANCED ENERGY INDS INC',
    'L3HARRIS TECHNOLOGIES INC',
    'SOLARWINDS CORP',
    'LOGITECH INTERNATIONAL SA',
    'CONDUENT INC',
    'THOUGHTWORKS HOLDG INC',
    'GLOBANT SA',
    'ELASTIC NV',
    'EPICOR SOFTWARE CORP -OLD',
    'SILICON MOTION TECH -ADR',
    'SPREADTRUM COMMUNICATNS -ADR',
    'RDA MICROELECTRONCS INC -ADR',
    'MATERIALISE NV',
    'DUCK CREEK TECHNOL INC',
    'YINGLI GREEN ENERGY HLDGS CO',
    'LINK MOTION INC -ADR',
]

reliable = crosswalk[
    (crosswalk['match_score'] >= 90) |
    (crosswalk['conm'].isin(manual_good_conm))
].copy()

print(f"Reliable matches: {len(reliable)}")
print(f" Auto-accepted (>= 90): {len(crosswalk[crosswalk['match_score'] >= 90])}")
print(f" Manually rescued (80-90): {len(reliable) - len(crosswalk[crosswalk['match_score'] >= 90])}")

# Ensure numeric columns
for col in ['Initial Approvals', 'Initial Denials',
            'Continuing Approvals', 'Continuing Denials']:
    h1b[col] = pd.to_numeric(h1b[col], errors='coerce').fillna(0).astype(int)

h1b_agg = h1b.groupby(['Fiscal Year', 'Employer']).agg({
    'Initial Approvals': 'sum',
    'Initial Denials': 'sum',
    'Continuing Approvals': 'sum',
    'Continuing Denials': 'sum',
}).reset_index()

```

```

h1b_agg['total_petitions'] = (h1b_agg['Initial Approvals'] + h1b_agg['Initial_
↳Denials'] +
                                h1b_agg['Continuing Approvals'] +_
↳h1b_agg['Continuing Denials'])
h1b_agg['total_approvals'] = (h1b_agg['Initial Approvals'] +_
↳h1b_agg['Continuing Approvals'])
h1b_agg['total_denials'] = (h1b_agg['Initial Denials'] + h1b_agg['Continuing_
↳Denials'])
h1b_agg['approval_rate'] = np.where(
    h1b_agg['total_petitions'] > 0,
    h1b_agg['total_approvals'] / h1b_agg['total_petitions'],
    0
)

print(f"H-1B employer-year observations: {len(h1b_agg)}")

# Attach gvkey to H-1B data through the crosswalk
h1b_matched = h1b_agg.merge(
    reliable[['gvkey', 'h1b_employer']],
    left_on='Employer', right_on='h1b_employer',
    how='inner'
)

print(f"H-1B matched firm-years: {len(h1b_matched)}")
print(f"H-1B matched unique firms: {h1b_matched['gvkey'].nunique()}")

# Left-join to Compustat panel (keep all Compustat firm-years)
final = panel.merge(
    h1b_matched.rename(columns={'Fiscal Year': 'fyear'}),
    on=['gvkey', 'fyear'],
    how='left'
)

# Fill NaN H-1B variables with 0 (firm had no H-1B activity that year)
h1b_fill_cols = ['Initial Approvals', 'Initial Denials',
                  'Continuing Approvals', 'Continuing Denials',
                  'total_petitions', 'total_approvals', 'total_denials']
for col in h1b_fill_cols:
    if col in final.columns:
        final[col] = final[col].fillna(0)

# --- Treatment definition for DiD ---
# Treatment group: firms that petitioned for H-1B in pre-period (before 2017 EO)
# The 2017 EO was buy american hire american
pre_h1b_firms = final[
    (final['fyear'] <= 2016) & (final['total_petitions'] > 0)

```

```

[['gvkey']].unique()
final['h1b_user_pre'] = final['gvkey'].isin(pre_h1b_firms).astype(int)

# Current-year H-1B indicator
final['h1b_user'] = (final['total_petitions'] > 0).astype(int)

# Post "Buy American, Hire American" executive order (April 2017)
final['post_baha'] = (final['fyear'] >= 2017).astype(int)

# DiD interaction term
final['h1b_user_x_post'] = final['h1b_user_pre'] * final['post_baha']

# --- Outcome variables ---
final['operating_margin'] = np.where(
    final['revt'] > 0, final['oibdp'] / final['revt'], np.nan
)
final['roa'] = final['ni'] / final['at']
final['rd_intensity'] = np.where(
    final['revt'] > 0, final['xrd'] / final['revt'], np.nan
)
final['log_emp'] = np.log(final['emp'].replace(0, np.nan))
final['log_revt'] = np.log(final['revt'].replace(0, np.nan))

# --- H-1B intensity ---
# Approvals per 1000 employees (emp is in thousands in Compustat)
final['h1b_intensity'] = np.where(
    final['emp'] > 0,
    final['total_approvals'] / (final['emp'] * 1000),
    0
)

final.to_csv('final_h1b_compustat_panel.csv', index=False)

print("\n" + "=" * 60)
print("FINAL PANEL SUMMARY")
print("=" * 60)
print(f"Total firm-years: {len(final)}")
print(f"Unique firms: {final['gvkey'].nunique()}")
print(f"Year range: {final['fyear'].min()} - {final['fyear'].max()}")

print(f"\nDiD groups:")
print(f"  Treatment (H-1B user pre-2017): □
    ↳ {final[final['h1b_user_pre']==1]['gvkey'].nunique()} firms")
print(f"  Control (no H-1B pre-2017): □
    ↳ {final[final['h1b_user_pre']==0]['gvkey'].nunique()} firms")

print(f"\nH-1B activity:")

```

```

print(f" Firm-years with petitions: {final['h1b_user'].sum()}")
print(f" Firm-years without:      {(~final['h1b_user']).astype(bool).sum()}")

print(f"\nOutcome variable means:")
for var in ['operating_margin', 'roa', 'rd_intensity', 'emp']:
    print(f" {var}: mean={final[var].mean():.4f}, sd={final[var].std():.4f}")

print(f"\nH-1B petition stats (conditional on > 0):")
active = final[final['total_petitions'] > 0]
print(f" N firm-years: {len(active)}")
print(f" Mean total petitions: {active['total_petitions'].mean():.1f}")
print(f" Mean approvals: {active['total_approvals'].mean():.1f}")
print(f" Mean approval rate: {active['approval_rate'].mean():.3f}")

```

Compustat panel: 2928 firm-years, 626 firms
 H-1B data: 457149 rows, 168947 unique employers
 Compustat years: 2010-2023
 H-1B fiscal years: 2009-2023
 Compustat firms to match: 626
 H-1B unique employers: 147698
 Running fuzzy matching (this may take a few minutes)...

Matching summary:

Score >= 90 (high confidence):	474
Score 80-90 (needs review):	91
Score 70-80 (likely wrong):	45
No match:	16

Reliable matches: 492

Auto-accepted (>= 90): 474

Manually rescued (80-90): 18

H-1B employer-year observations: 398819

H-1B matched firm-years: 3828

H-1B matched unique firms: 492

=====

FINAL PANEL SUMMARY

=====

Total firm-years: 2928

Unique firms: 626

Year range: 2010 - 2023

DiD groups:

Treatment (H-1B user pre-2017): 251 firms

Control (no H-1B pre-2017): 375 firms

H-1B activity:

Firm-years with petitions: 1474

Firm-years without: 1454

Outcome variable means:

operating_margin: mean=-0.4407, sd=18.9578
roa: mean=0.0164, sd=0.1479
rd_intensity: mean=0.2310, sd=2.1151
emp: mean=8.9959, sd=22.2881

H-1B petition stats (conditional on > 0):

N firm-years: 1474
Mean total petitions: 30.8
Mean approvals: 29.9
Mean approval rate: 0.980

```
[22]: final_data = pd.read_csv('final_h1b_compustat_panel.csv')
```

```
[23]: final_data.describe()
```

```
[23]:
```

	gvkey	fyear	naicsh	sich	emp	\
count	2928.000000	2928.000000	2928.000000	2912.000000	2886.000000	
mean	81415.150273	2016.221311	406644.330601	5268.265453	8.995851	
std	70991.256822	4.081950	115016.744422	1841.533247	22.288120	
min	1013.000000	2010.000000	3341.000000	2711.000000	0.001000	
25%	21694.750000	2013.000000	334413.000000	3674.000000	1.394000	
50%	39436.500000	2016.000000	334515.000000	3845.000000	3.041500	
75%	156153.000000	2020.000000	518210.000000	7372.000000	7.222500	
max	300107.000000	2023.000000	541519.000000	8200.000000	440.000000	

	at	revt	oibdp	ni	xrd	...	\
count	2928.000000	2927.000000	2927.000000	2927.000000	2502.000000	...	
mean	2392.274777	1728.668920	222.91947	31.016240	139.147148	...	
std	3258.869088	3007.108645	352.82652	336.247722	182.229879	...	
min	0.006000	0.000000	-2214.00000	-5873.000000	0.000000	...	
25%	670.816750	425.546000	46.40650	-20.283000	45.171750	...	
50%	1387.646500	854.637000	135.30000	44.400000	90.501500	...	
75%	2754.438000	1820.440500	301.09550	119.631000	168.137750	...	
max	32032.242000	33478.000000	5651.20400	3675.000000	3222.658000	...	

	h1b_user_pre	h1b_user	post_baha	h1b_user_x_post	\
count	2928.000000	2928.000000	2928.000000	2928.000000	
mean	0.535861	0.503415	0.466872	0.172814	
std	0.498798	0.500074	0.498987	0.378151	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	
50%	1.000000	1.000000	0.000000	0.000000	
75%	1.000000	1.000000	1.000000	0.000000	
max	1.000000	1.000000	1.000000	1.000000	

	operating_margin	roa	rd_intensity	log_emp	log_revt \
count	2922.000000	2927.000000	2499.000000	2886.000000	2922.000000
mean	-0.440674	0.016440	0.230951	1.200740	6.784883
std	18.957797	0.147853	2.115146	1.333487	1.159482
min	-867.800000	-3.076495	0.000000	-6.907755	-3.912023
25%	0.072826	-0.017169	0.066354	0.332177	6.053671
50%	0.150372	0.035421	0.135042	1.112347	6.752549
75%	0.215901	0.075738	0.204893	1.977200	7.507912
max	0.716590	0.715750	73.200000	6.086775	10.418644

	h1b_intensity
count	2928.000000
mean	0.004522
std	0.009401
min	0.000000
25%	0.000000
50%	0.000000
75%	0.004233
max	0.072131

[8 rows x 41 columns]

```
[24]: print("=" * 70)
print("DATASET OVERVIEW")
print("=" * 70)
print(f"Observations (firm-years): {len(final_data)}")
print(f"Unique firms: {final_data['gvkey'].nunique()}")
print(f"Year range: {final_data['fyear'].min()} - {final_data['fyear'].max()}")
```

```
=====
DATASET OVERVIEW
=====
Observations (firm-years): 2928
Unique firms: 626
Year range: 2010 - 2023
```

```
[25]: variable_guide = pd.DataFrame([
    # Identifiers
    ('gvkey', 'Identifier', 'Compustat permanent firm ID'),
    ('comm', 'Identifier', 'Company name'),
    ('cusip', 'Identifier', 'CUSIP security identifier'),
    ('fyear', 'Identifier', 'Fiscal year'),
    ('naicsh', 'Identifier', 'NAICS industry code (historical)'),
    ('sich', 'Identifier', 'SIC industry code (historical)'),

    # Firm size
```

```

    ('emp',                'Firm Size',      'Employees in thousands (9.3 = 9,300 workers)'),
    ('at',                 'Firm Size',      'Total assets ($M)'),
    ('revt',               'Firm Size',      'Total revenue ($M)'),
    ('mktcap',             'Firm Size',      'Market capitalization ($M) = stock price × shares'),
    ('csho',               'Firm Size',      'Common shares outstanding (millions)'),
    ('prcc_f',             'Firm Size',      'Stock price at fiscal year-end ($)'),

    # Profitability
    ('oibdp',              'Profitability', 'Operating income before depreciation / EBITDA ($M)'),
    ('ni',                 'Profitability', 'Net income ($M)'),
    ('operating_margin',   'Profitability', 'Operating margin = oibdp / revt'),
    ('roa',               'Profitability', 'Return on assets = ni / at'),

    # Investment & costs
    ('xrd',                'Investment',     'R&D expenditure ($M)'),
    ('rd_intensity',       'Investment',     'R&D intensity = xrd / revt'),
    ('xsga',               'Investment',     'Selling, general & admin expense ($M)'),
    ('capx',               'Investment',     'Capital expenditure ($M)'),

    # Balance sheet
    ('lt',                 'Balance Sheet',  'Total liabilities ($M)'),
    ('ceq',                'Balance Sheet',  'Common equity ($M)'),
    ('che',                'Balance Sheet',  'Cash & short-term investments ($M)'),

    # H-1B petition data
    ('Initial Approvals',  'H-1B Data',      'New H-1B petitions approved this fiscal year'),
    ('Initial Denials',    'H-1B Data',      'New H-1B petitions denied'),
    ('Continuing Approvals', 'H-1B Data',      'H-1B renewals/extensions approved'),
    ('Continuing Denials', 'H-1B Data',      'H-1B renewals/extensions denied'),
    ('total_petitions',    'H-1B Data',      'Total petitions filed (approvals + denials)'),
    ('total_approvals',    'H-1B Data',      'Total approved (initial + continuing)'),
    ('total_denials',      'H-1B Data',      'Total denied (initial + continuing)'),
    ('approval_rate',      'H-1B Data',      'Approval rate = total_approvals / total_petitions'),

```

```

    ('h1b_intensity',      'H-1B Data',      'H-1B approvals per employee'),

    # DiD variables
    ('h1b_user',          'DiD Design',      '1 if firm filed any H-1B petition_
↪this year'),
    ('h1b_user_pre',      'DiD Design',      '1 if firm filed any H-1B petition_
↪before 2017 (TREATMENT GROUP)'),
    ('post_baha',         'DiD Design',      '1 if fiscal year >= 2017_
↪(POST-PERIOD)'),
    ('h1b_user_x_post',   'DiD Design',      'Interaction: h1b_user_pre ×_
↪post_baha (DiD COEFFICIENT)'),

    # Constructed
    ('log_emp',           'Constructed',      'Natural log of employees'),
    ('log_revt',          'Constructed',      'Natural log of revenue'),
], columns=['Variable', 'Category', 'Description'])

print("\n" + "=" * 70)
print("VARIABLE DICTIONARY")
print("=" * 70)
for cat in variable_guide['Category'].unique():
    print(f"\n--- {cat} ---")
    cat_vars = variable_guide[variable_guide['Category'] == cat]
    for _, row in cat_vars.iterrows():
        print(f"  {row['Variable']:25s} {row['Description']}")

stats_vars = [
    # Firm size
    ('emp',               'Employees (000s)'),
    ('revt',               'Revenue ($M)'),
    ('at',                 'Total Assets ($M)'),
    ('mktcap',             'Market Cap ($M)'),
    # Profitability
    ('ni',                 'Net Income ($M)'),
    ('oibdp',              'EBITDA ($M)'),
    ('roa',                 'Return on Assets'),
    ('operating_margin',   'Operating Margin'),
    # Investment
    ('xrd',                 'R&D Expense ($M)'),
    ('rd_intensity',       'R&D / Revenue'),
    ('capx',               'Capital Expenditure ($M)'),
    ('che',                 'Cash Holdings ($M)'),
    # H-1B
    ('total_petitions',    'H-1B Petitions'),
    ('total_approvals',    'H-1B Approvals'),
    ('total_denials',      'H-1B Denials'),
    ('approval_rate',      'H-1B Approval Rate'),

```

```

        ('h1b_intensity',      'H-1B per Employee'),
    ]

summary_rows = []
for var, label in stats_vars:
    if var in final_data.columns:
        s = final_data[var]
        summary_rows.append({
            'Variable': label,
            'N': int(s.notna().sum()),
            'Mean': s.mean(),
            'SD': s.std(),
            'P25': s.quantile(0.25),
            'Median': s.median(),
            'P75': s.quantile(0.75),
        })

summary_df = pd.DataFrame(summary_rows)

print("\n" + "=" * 70)
print("PANEL A: SUMMARY STATISTICS - FULL SAMPLE")
print("=" * 70)
print(summary_df.to_string(index=False, float_format='{: .3f}'.format))

print("\n" + "=" * 70)
print("PANEL B: TREATMENT vs CONTROL (Pre-2017 H-1B users vs Non-users)")
print("=" * 70)

treat = final_data[final_data['h1b_user_pre'] == 1]
ctrl = final_data[final_data['h1b_user_pre'] == 0]

comparison_rows = []
for var, label in stats_vars:
    if var in final_data.columns:
        t = treat[var].dropna()
        c = ctrl[var].dropna()
        comparison_rows.append({
            'Variable': label,
            'Treatment Mean': t.mean(),
            'Control Mean': c.mean(),
            'Difference': t.mean() - c.mean(),
        })

comparison_df = pd.DataFrame(comparison_rows)
print(comparison_df.to_string(index=False, float_format='{: .3f}'.format))

print("\n" + "=" * 70)

```

```

print("PANEL C: DiD GROUP STRUCTURE")
print("=" * 70)

did_table = final_data.groupby(['h1b_user_pre', 'post_baha']).agg(
    firm_years=('gvkey', 'count'),
    unique_firms=('gvkey', 'nunique'),
    mean_roa=('roa', 'mean'),
    mean_petitions=('total_petitions', 'mean'),
).reset_index()

did_table['h1b_user_pre'] = did_table['h1b_user_pre'].map({0: 'Control', 1: 'Treatment'})
did_table['post_baha'] = did_table['post_baha'].map({0: 'Pre-BAHA (2010-2016)', 1: 'Post-BAHA (2017-2023)'})
did_table.columns = ['Group', 'Period', 'Firm-Years', 'Unique Firms', 'Mean ROA', 'Mean H-1B Petitions']

print(did_table.to_string(index=False, float_format='{: .4f}'.format))

print("\n" + "=" * 70)
print("PANEL D: YEAR-BY-YEAR FIRM COUNTS")
print("=" * 70)
yearly = final_data.groupby(['fyear', 'h1b_user_pre'])['gvkey'].nunique().unstack(fill_value=0)
yearly.columns = ['Control', 'Treatment']
yearly['Total'] = yearly.sum(axis=1)
print(yearly.to_string())

```

===== VARIABLE DICTIONARY =====

--- Identifier ---

gvkey	Compustat permanent firm ID
conm	Company name
cusip	CUSIP security identifier
fyear	Fiscal year
naicsh	NAICS industry code (historical)
sich	SIC industry code (historical)

--- Firm Size ---

emp	Employees in thousands (9.3 = 9,300 workers)
at	Total assets (\$M)
revt	Total revenue (\$M)
mktpcap	Market capitalization (\$M) = stock price × shares
csho	Common shares outstanding (millions)

prcc_f	Stock price at fiscal year-end (\$)
--- Profitability ---	
oibdp	Operating income before depreciation / EBITDA (\$M)
ni	Net income (\$M)
operating_margin	Operating margin = oibdp / revt
roa	Return on assets = ni / at
--- Investment ---	
xrd	R&D expenditure (\$M)
rd_intensity	R&D intensity = xrd / revt
xsga	Selling, general & admin expense (\$M)
capx	Capital expenditure (\$M)
--- Balance Sheet ---	
lt	Total liabilities (\$M)
ceq	Common equity (\$M)
che	Cash & short-term investments (\$M)
--- H-1B Data ---	
Initial Approvals	New H-1B petitions approved this fiscal year
Initial Denials	New H-1B petitions denied
Continuing Approvals	H-1B renewals/extensions approved
Continuing Denials	H-1B renewals/extensions denied
total_petitions	Total petitions filed (approvals + denials)
total_approvals	Total approved (initial + continuing)
total_denials	Total denied (initial + continuing)
approval_rate	Approval rate = total_approvals / total_petitions
h1b_intensity	H-1B approvals per employee
--- DiD Design ---	
h1b_user	1 if firm filed any H-1B petition this year
h1b_user_pre	1 if firm filed any H-1B petition before 2017
(TREATMENT GROUP)	
post_baha	1 if fiscal year >= 2017 (POST-PERIOD)
h1b_user_x_post	Interaction: h1b_user_pre × post_baha (DiD
COEFFICIENT)	
--- Constructed ---	
log_emp	Natural log of employees
log_revt	Natural log of revenue

```
=====
PANEL A: SUMMARY STATISTICS - FULL SAMPLE
=====
```

	Variable	N	Mean	SD	P25	Median	P75
	Employees (000s)	2886	8.996	22.288	1.394	3.042	7.223
	Revenue (\$M)	2927	1728.669	3007.109	425.546	854.637	1820.440

Total Assets (\$M)	2928	2392.275	3258.869	670.817	1387.646	2754.438
Market Cap (\$M)	2928	3008.433	2066.113	1439.734	2350.587	4023.602
Net Income (\$M)	2927	31.016	336.248	-20.283	44.400	119.631
EBITDA (\$M)	2927	222.919	352.827	46.406	135.300	301.096
Return on Assets	2927	0.016	0.148	-0.017	0.035	0.076
Operating Margin	2922	-0.441	18.958	0.073	0.150	0.216
R&D Expense (\$M)	2502	139.147	182.230	45.172	90.501	168.138
R&D / Revenue	2499	0.231	2.115	0.066	0.135	0.205
Capital Expenditure (\$M)	2925	89.024	280.698	11.311	27.215	63.181
Cash Holdings (\$M)	2928	462.428	555.572	147.474	294.850	548.099
H-1B Petitions	2928	15.527	67.287	0.000	1.000	11.000
H-1B Approvals	2928	15.028	64.442	0.000	1.000	11.000
H-1B Denials	2928	0.499	3.495	0.000	0.000	0.000
H-1B Approval Rate	1474	0.980	0.054	0.983	1.000	1.000
H-1B per Employee	2928	0.005	0.009	0.000	0.000	0.004

=====

PANEL B: TREATMENT vs CONTROL (Pre-2017 H-1B users vs Non-users)

=====

Variable	Treatment Mean	Control Mean	Difference
Employees (000s)	7.504	10.737	-3.233
Revenue (\$M)	1502.955	1989.453	-486.497
Total Assets (\$M)	1911.026	2947.889	-1036.863
Market Cap (\$M)	2705.203	3358.519	-653.316
Net Income (\$M)	60.557	-3.114	63.670
EBITDA (\$M)	201.032	248.208	-47.176
Return on Assets	0.039	-0.010	0.049
Operating Margin	0.089	-1.055	1.144
R&D Expense (\$M)	141.482	136.204	5.278
R&D / Revenue	0.201	0.269	-0.068
Capital Expenditure (\$M)	62.730	119.448	-56.719
Cash Holdings (\$M)	422.718	508.275	-85.556
H-1B Petitions	23.972	5.776	18.196
H-1B Approvals	23.205	5.587	17.618
H-1B Denials	0.767	0.189	0.578
H-1B Approval Rate	0.980	0.979	0.001
H-1B per Employee	0.007	0.002	0.005

=====

PANEL C: DiD GROUP STRUCTURE

=====

Group	Period	Firm-Years	Unique Firms	Mean ROA	Mean H-1B
Petitions					
Control	Pre-BAHA (2010-2016)	498	165	0.0248	
0.0000					
Control	Post-BAHA (2017-2023)	861	279	-0.0300	
9.1173					
Treatment	Pre-BAHA (2010-2016)	1063	251	0.0389	

```

24.2531
Treatment Post-BAHA (2017-2023)          506          127    0.0399
23.3814

```

```

=====
PANEL D: YEAR-BY-YEAR FIRM COUNTS
=====

```

	Control	Treatment	Total
fyear			
2010	79	168	247
2011	70	164	234
2012	61	159	220
2013	73	160	233
2014	72	147	219
2015	77	137	214
2016	66	128	194
2017	74	107	181
2018	80	97	177
2019	109	76	185
2020	149	67	216
2021	183	62	245
2022	137	52	189
2023	129	45	174

```

[26]: print(final_data.columns)
      print(len(final_data.columns))

Index(['gvkey', 'conm', 'cusip', 'fyear', 'naicsh', 'sich', 'emp', 'at',
      'revt', 'oibdp', 'ni', 'xrd', 'xsga', 'xlr', 'lt', 'ceq', 'capx', 'che',
      'prcc_f', 'csho', 'mktcap', 'mktcap_rank', 'r2000_proxy', 'naicsh_str',
      'tech', 'Employer', 'Initial Approvals', 'Initial Denials',
      'Continuing Approvals', 'Continuing Denials', 'total_petitions',
      'total_approvals', 'total_denials', 'approval_rate', 'hib_employer',
      'hib_user_pre', 'hib_user', 'post_baha', 'hib_user_x_post',
      'operating_margin', 'roa', 'rd_intensity', 'log_emp', 'log_revt',
      'hib_intensity'],
      dtype='object')

```

45

```

[27]: import matplotlib.pyplot as plt
      import matplotlib.ticker as mticker
      from scipy import stats

```

```

[28]: TREAT_COLOR = '#2166ac'    # blue for H-1B users
      CTRL_COLOR = '#b2182b'    # red for non-users
      ACCENT = '#4daf4a'        # green accent

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

```



```

yearly_counts = final_data.groupby(['fyear', 'h1b_user_pre'])['gvkey'].
    ↪nunique().unstack(fill_value=0)
yearly_counts.columns = ['Control', 'Treatment']
yearly_counts.plot(kind='bar', stacked=True, ax=axes[0],
                    color=[CTRL_COLOR, TREAT_COLOR], alpha=0.8, width=0.75)
axes[0].set_title('(a) Firm Count by Year and H-1B Status')
axes[0].set_xlabel('Fiscal Year')
axes[0].set_ylabel('Number of Firms')
axes[0].legend(title='Pre-2017 H-1B User')
axes[0].axvline(x=6.5, color='black', linestyle='--', alpha=0.5, label='BAHA EO_
    ↪(2017)')
axes[0].tick_params(axis='x', rotation=45)

h1b_by_year = final_data[final_data['total_petitions'] > 0].groupby('fyear').
    ↪agg(
        firms_petitioning=('gvkey', 'nunique'),
        mean_petitions=('total_petitions', 'mean'),
        total_petitions=('total_petitions', 'sum'),
    ).reindex(range(2010, 2024), fill_value=0)

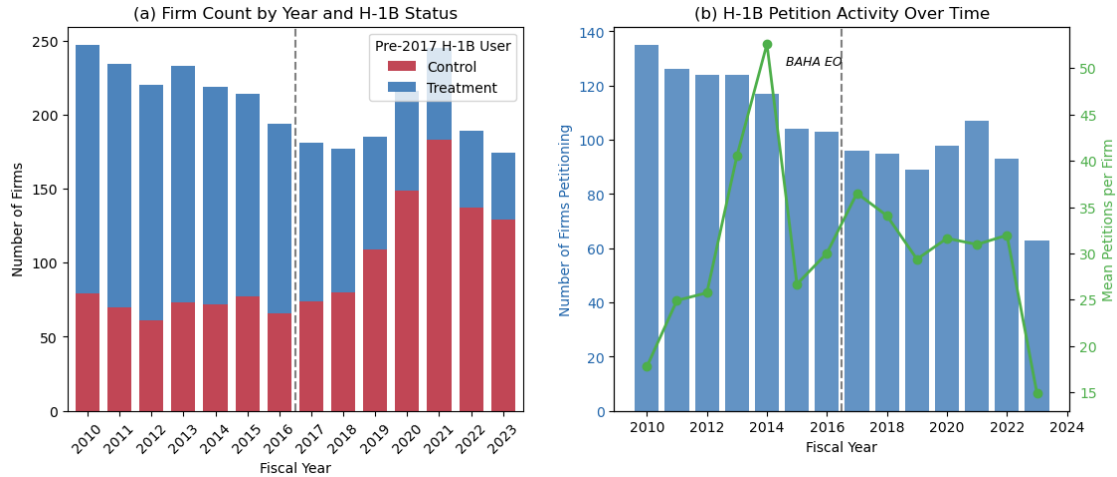
ax1b = axes[1]
bars = ax1b.bar(h1b_by_year.index, h1b_by_year['firms_petitioning'],
                color=TREAT_COLOR, alpha=0.7, label='Firms petitioning')
ax1b.set_xlabel('Fiscal Year')
ax1b.set_ylabel('Number of Firms Petitioning', color=TREAT_COLOR)
ax1b.tick_params(axis='y', labelcolor=TREAT_COLOR)

ax1b_twin = ax1b.twinx()
ax1b_twin.plot(h1b_by_year.index, h1b_by_year['mean_petitions'],
                color=ACCENT, marker='o', linewidth=2, label='Mean petitions per_
    ↪firm')
ax1b_twin.set_ylabel('Mean Petitions per Firm', color=ACCENT)
ax1b_twin.tick_params(axis='y', labelcolor=ACCENT)

ax1b.axvline(x=2016.5, color='black', linestyle='--', alpha=0.5)
ax1b.set_title('(b) H-1B Petition Activity Over Time')
ax1b.annotate('BAHA EO', xy=(2016.5, ax1b.get_ylim()[1]*0.9),
              fontsize=9, ha='right', style='italic')

```

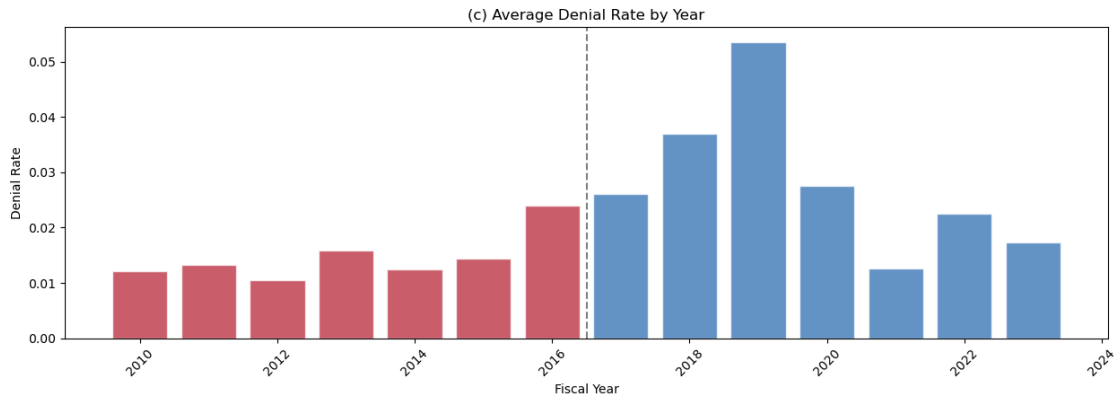
[28]: Text(2016.5, 127.575, 'BAHA EO')



```
[32]: fig, axes = plt.subplots(1, 1, figsize=(15, 4.5))

denial_by_year = active.groupby('fyear')['approval_rate'].agg(['mean', 'std', 'count'])
denial_by_year['denial_rate'] = 1 - denial_by_year['mean']
denial_by_year['se'] = denial_by_year['std'] / np.sqrt(denial_by_year['count'])

axes.bar(denial_by_year.index, denial_by_year['denial_rate'],
         color=[CTRL_COLOR if y < 2017 else TREAT_COLOR for y in denial_by_year.index],
         alpha=0.7, edgecolor='white')
axes.set_title('(c) Average Denial Rate by Year')
axes.set_xlabel('Fiscal Year')
axes.set_ylabel('Denial Rate')
axes.axvline(x=2016.5, color='black', linestyle='--', alpha=0.5)
axes.tick_params(axis='x', rotation=45)
```



```
[33]: import statsmodels.api as sm
      from scipy import stats
```

```
[34]: def winsorize(s, lower=0.01, upper=0.99):
      q_lo, q_hi = s.quantile([lower, upper])
      return s.clip(q_lo, q_hi)

final_data['roa_w'] = winsorize(final_data['roa'].dropna()).reindex(final_data.
    ↪index)
final_data['op_margin_w'] = winsorize(final_data['operating_margin'].dropna()).
    ↪reindex(final_data.index)
final_data['rd_intensity_w'] = winsorize(final_data['rd_intensity'].dropna()).
    ↪reindex(final_data.index)
final_data['log_petitions'] = np.log1p(final_data['total_petitions'])
final_data['log_approvals'] = np.log1p(final_data['total_approvals'])

# Only firm-years with H-1B activity for scatter plots
active = final_data[final_data['total_petitions'] > 0].copy()

print(f"Full sample: {len(final_data)} firm-years")
print(f"H-1B active: {len(active)} firm-years")
```

Full sample: 2928 firm-years

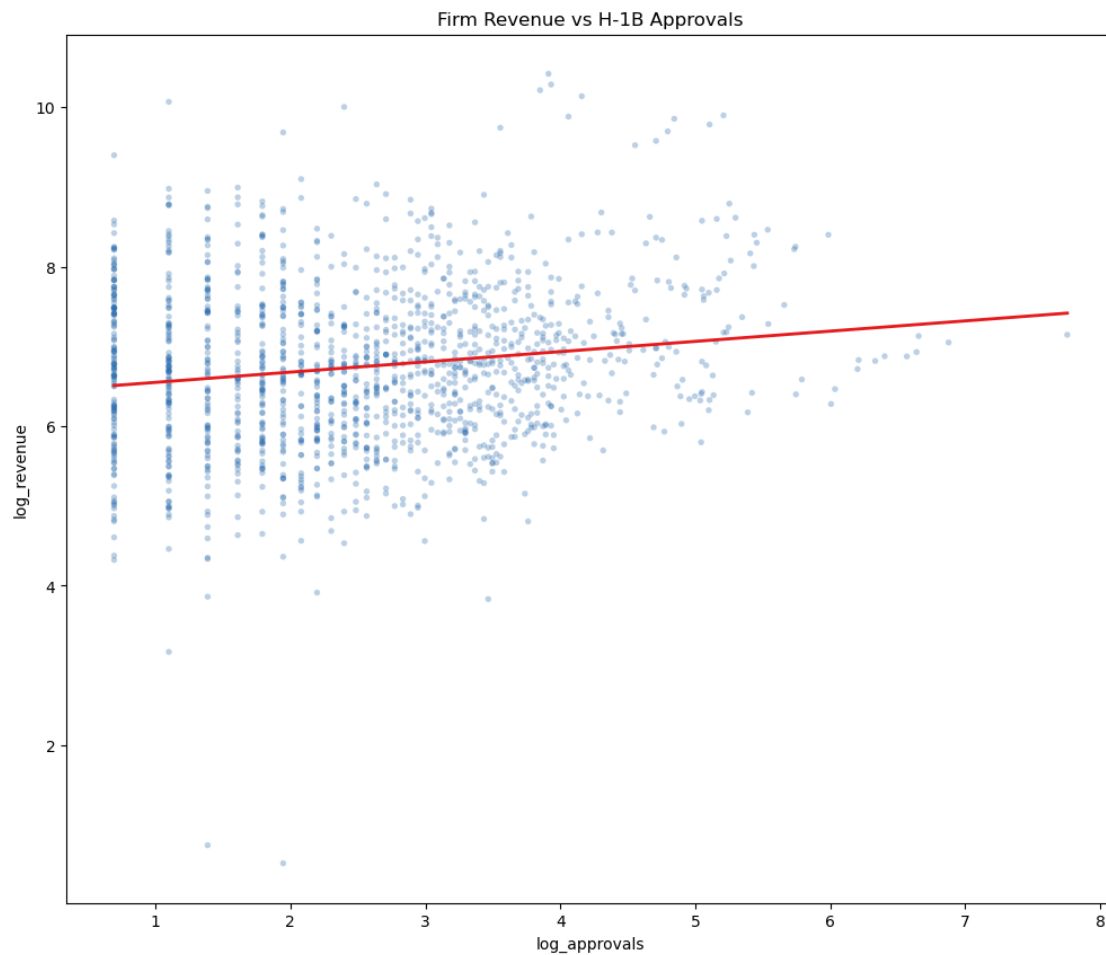
H-1B active: 1474 firm-years

```
[39]: valid = active[['log_approvals', 'log_revt']].dropna()
fig, ax = plt.subplots(1, 1, figsize=(12, 10))
ax.scatter(valid['log_approvals'], valid['log_revt'], alpha=0.3, s=15,
    ↪color='#2166ac', edgecolors='none')

# OLS fit line
X = sm.add_constant(valid['log_approvals'])
model = sm.OLS(valid['log_revt'], X).fit()
x_line = np.linspace(valid['log_approvals'].min(), valid['log_approvals'].
    ↪max(), 100)
y_line = model.predict(sm.add_constant(x_line))
ax.plot(x_line, y_line, color='#e41a1c', linewidth=2)

ax.set_xlabel('log_approvals')
ax.set_ylabel('log_revenue')
ax.set_title('Firm Revenue vs H-1B Approvals')
```

```
[39]: Text(0.5, 1.0, 'Firm Revenue vs H-1B Approvals')
```



```
[41]: print(model.summary())
```

```

OLS Regression Results
=====
Dep. Variable:          log_revt    R-squared:                0.024
Model:                  OLS         Adj. R-squared:           0.023
Method:                 Least Squares   F-statistic:             36.29
Date:                  Fri, 20 Mar 2026   Prob (F-statistic):      2.14e-09
Time:                  21:33:45         Log-Likelihood:          -2080.1
No. Observations:      1474           AIC:                    4164.
Df Residuals:          1472           BIC:                    4175.
Df Model:               1
Covariance Type:       nonrobust
=====
=
               coef      std err          t      P>|t|      [0.025
0.975]
-----
-----

```

```

-
const          6.4194      0.060    106.957      0.000      6.302
6.537
log_approvals  0.1286      0.021      6.024      0.000      0.087
0.170
=====
Omnibus:                65.732    Durbin-Watson:                1.773
Prob(Omnibus):           0.000    Jarque-Bera (JB):            209.489
Skew:                    0.037    Prob(JB):                     3.24e-46
Kurtosis:                4.845    Cond. No.                      7.21
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[]: